Nordwerk Treasury

Platform Implementation Report

Exception Management, Risk Intelligence & Treasury Operations

Complete technical overview, functionality catalogue, KPI definitions, SAP integration position, deployment notes and end-to-end demonstration script.

Prepared for local demonstration

15 October 2026

Version: demo implementation · Data source: seeded, in-memory demo data

1. Executive summary

Nordwerk Treasury Platform is a web application for identifying, prioritising and resolving treasury-process exceptions. It consolidates document-level exceptions—such as purchase-order approval gaps, payment mismatches, missing master data and aged cash-flow items—into a single operational queue. Users can assess risk, validate data, assign or escalate ownership, resolve exceptions, inspect audit history, analyse portfolio trends and distribute outputs as PDF reports or email.

The delivered implementation is designed as a realistic local demonstration. It exposes a REST API through Express, presents an SAP Fiori-inspired single-page user interface, and runs against seeded in-memory data. It also includes a CAP/CDS domain and service model that documents the target enterprise data design. This separation makes the demo usable immediately while retaining a clear migration path to persistent storage and live SAP services.

Business outcomes

- A single prioritised work queue for treasury exceptions across purchase orders, invoices, payments and cash-flow records.
- Risk-aware resolution: financial exposure, aging, data completeness, recurrence and document criticality are combined into a transparent score.
- Improved control and traceability through validation, timeline events, resolution logs and an audit trail.
- Decision-ready reporting through live KPIs, analytical visualisations, PDF export and email delivery.

Scope and operating model

Current data mode: In-memory seeded demo data. Changes last until the backend process is restarted.

Primary users: Treasury analyst, treasury manager, finance/CFO stakeholder and administrator personas represented in the CDS model.

Deployment shape: Local Express service during development; Netlify Function wrapper and SPA redirects for deployment.

2. Architecture and technology stack

The application has a lightweight three-layer architecture. The browser renders a Vite-based JavaScript SPA. It calls the Express API over HTTP. The API applies validation, scoring, analytics, document generation, audit and notification logic over the demo store. The same Express application is exported to a serverless handler for Netlify.

Layer	Technology	Role in the implementation
Frontend	Vite 8, vanilla ES modules, HTML/CSS	Single-page UI with Dashboard, Exception Queue and Analytics pages; Fiori-inspired styling and SVG charts.
Backend	Node.js 22, Express 5	REST API, business workflow, data filtering, security middleware and local HTTP listener on port 4004.
Enterprise model	SAP CAP / CDS	Defines target entities, associations, service projections, actions and analytics/KPI contracts.
Risk & AI	Rule engine + deterministic explainer	Computes risk/SLA/priority and produces explainable next-best-action guidance without an external AI dependency.
Documents	PDFKit	Creates the downloadable landscape A4 exception report directly from filtered API data.
Email	Nodemailer	Uses Ethereal preview inboxes by default; supports real SMTP credentials through environment variables.
Security	Helmet, CORS, express-rate-limit	Sets response protections, cross-origin policy and API request limiting.

Layer	Technology	Role in the implementation
Deployment	Netlify, serverless-http	Wraps the Express app as a Netlify Function and rewrites /api/* to it.
Quality	Node test runner	32 validation, risk and explainer tests currently pass.

Key packages

Backend dependencies include @sap/cds and @sap/cds-dk, Express, CORS, Hel
date-fns, PDFKit, Nodemailer, ExcelJS, Multer and serverless-http. The frontend us
Node.js 22 in the Netlify configuration.

3. Data model and operational concept

The CAP/CDS schema defines the enterprise-ready information model. TreasuryEx
associated with CompanyCodes, Plants, CostCentres, Materials, BusinessPartners
be understood in its financial and operational context.

Concept	What is stored / why it matters
Exception identity	Document ID, document type, company code, plant, business partner, amount, currency and relevant dates.
Classification	Status, category, subcategory, risk level, risk score and queue priority.
SLA and aging	Raised date, calculated aging days, SLA deadline, breach flag and overdue value.
Ownership	Assigned owner, assignment time, escalation target and escalation time.
Resolution	Resolver, resolution timestamp, notes and approval state.
Governance	Validation messages, timeline events, resolution actions, notifications, attachments and append-only audit records.
Process intelligence	Signavio process step and process-deviation description.

The current API initialises its demo store from backend/db/seed-data/exceptions.json. It now contains
escalated items and three resolved historical items. The resolved records intentionally supply raisedA
resolvedAt timestamps so resolution-cycle and average-resolution metrics are meaningful in a demon

4. Functional implementation catalogue

4.1 Executive Dashboard

- Live API status indicator confirms whether the frontend can reach the backend.
- Total Exceptions shows the all-time count and open/resolved split.
- Total Exposure sums the value of open exceptions; overdue exposure is highlighted when applicable.
- SLA Adherence measures open exceptions that are within SLA.
- Average Resolution shows mean elapsed hours for resolved exceptions with valid raised and resolved timestamps.
- Aging Buckets group open items into 0–3, 3–7, 7–14 and over-14-day bands.

- Risk Distribution shows open exceptions by Critical, High, Medium and Low severity.
- Recent High-Priority Exceptions presents the items most in need of action and links to the full queue.

4.2 Exception Queue

- Filters by status, risk level, document type and free-text search across document ID, partner and category.
- Supports server-side sorting and pagination with OData-style $top and $skip query parameters.
- Assessment opens a detailed risk panel containing score, SLA information, rule breakdown, AI explanation and recommended action.
- Resolve captures resolution notes (minimum ten characters), sets the status to Resolved, timestamps the action and records audit/timeline entries.
- Resolve Selected supports batch selection in the work queue; refresh reloads current API data.
- Export PDF respects the current filter query and downloads a branded exception report.
- Email Report sends a queue summary. A risk-assessment panel can also email the selected exception detail.

4.3 Analytics

- Category/root-cause analysis counts exceptions and their associated exposure.
- Business-partner analysis ranks partners by total exception value.
- Risk donut summarises the severity profile of open work.
- Resolution Cycle Time charts the elapsed hours for resolved cases.
- Plant/location chart shows the concentration of exceptions by plant.

5. Backend workflow and API functions

The REST API is rooted at /api/treasury. Responses use JSON except the PDF export. The API applies Helmet, JSON body parsing, CORS and a 200-requests-per-minute limiter for /api/ routes. Each request receives an X-Request-ID or generated UUID for traceability.

Endpoint group	Implemented functions
Exceptions	List/filter, detail, create and PATCH update of /exceptions.
Workflow	Validate, assign owner, escalate, resolve and add comment for a selected exception.
Risk / intelligence	Detailed risk endpoint and explainable AI analysis endpoint.
Management insight	Portfolio KPI endpoint and analytics endpoint for root causes, partners, locations and cycle times.
Traceability	Timeline, resolution log and audit-log retrieval.
Data movement	Bulk JSON import, PDF export and email dispatch.
Health	Health endpoint confirming service status, seeded exception count and SAP mode.

Validation engine

The validation handler checks mandatory fields, identifier formats, monetary ranges, date ordering consistency, self-approval, missing resolution notes, missing escalation target and payment-misma returns structured errors, warnings, information messages, a plain-language summary and a comm of missing fields. Validation is run during seed initialisation, record creation and relevant workflow a

Risk and next-best-action engine

Risk is transparent and deterministic. The score combines financial exposure (35%), aging (30%), data completeness (20%), recurrence (10%) and document criticality (5%). It calculates SLA deadline, hours overdue, overdue value and priority. The AI explainer then maps evidence to a concise explanation, confidence value and recommended action—such as field correction, owner assignment, approval request, escalation or SAP API call. It is explainable business logic rather than a call to a generative model.

6. KPI catalogue and calculation definitions

KPI	Definition / business interpretation
Exception Count	All current records in the portfolio, regardless of status.
Open Count	Items not in Resolved, Closed or Cancelled status.
Resolved Count	Items whose status is Resolved.
Total Exposure	Sum of amount for open items; indicates financial value requiring attention.
Overdue Total	Sum of amount for open items whose calculated SLA is breached.
SLA Adherence	Open items not breached ÷ all open items × 100.
Average Resolution	Mean hours from raisedAt to resolvedAt over resolved records.
First-Pass Yield	Resolved items with recurrenceCount = 0 ÷ all exceptions × 100.
Recurrence Rate	Exceptions with recurrenceCount > 0 ÷ all exceptions × 100.
Revenue at Risk	Open exposure ÷ configured annual revenue × 100.
Aging Buckets	Count of open exceptions grouped by age band for workload management.
Risk Distribution	Count of open exceptions in each severity level.

The dashboard deliberately treats resolved records differently from open exposure. Resolved item and resolution-quality metrics but are excluded from active financial exposure and SLA adherence historical completions from masking current operational risk.

7. SAP and process-integration position

The application is SAP-aligned but the current demo does not connect to a live SAP system. This important for a correct demonstration. The CDS model uses SAP-style enterprise entities, OData- definitions and SAP key fields (company code, plant, cost centre, material, business partner and c identifiers). It also records a Signavio process step and deviation per exception.

SAP release function implemented in the demo

- Endpoint: POST /api/treasury/exceptions/:id/sap-release.
- Purpose: simulate a controlled release of the selected business document after an exception is resolved or approved.

- Payload fields: DocumentId, DocumentType, CompanyCode, Amount, Currency, Status=Released, ChangedBy and ChangedAt.
- Safety gate: for amounts above £50,000, the API returns HTTP 202 and requires explicit confirmation before the simulated call proceeds.
- Result: a mocked SAP response is stored in the resolution log, an audit entry and a timeline event are created, and successful calls move the item to InProgress.
- Transaction reference: the simulation returns ME29N for a PO and F-53 for other document types as a familiar operational reference.

What is not yet a live SAP integration

No SAP BAPI, RFC destination, SAP S/4HANA OData URL, service key or OAuth flow is configured in this repository. The SAP release is explicitly simulated and can fail at a configurable demonstration rate. To productionise it, replace simulateSAPCall with a secure destination-backed SAP integration, use a real business-object API, validate authorisations, map SAP error messages, implement retries/idempotency and persist integration logs.

8. PDF export and email delivery

PDF export

GET /api/treasury/export/pdf generates a landscape A4 document using PDFKit. It includes a branded header, generation date, portfolio KPI strip and a tabular list of up to 35 filtered exceptions. The browser action passes the active queue filters, so the downloaded report corresponds to the visible operational context. The endpoint has been verified locally to return HTTP 200, Content-Type application/pdf and a date-stamped attachment filename.

Email

POST /api/treasury/mail sends either a queue summary or a rich selected-exception table. Nodemailer creates an Ethereal test account when SMTP settings are absent; this safely returns an inbox-preview URL for demonstrations. For real delivery, create backend/.env from backend/.env.example and set SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS and MAIL_FROM. The start and dev scripts load this .env automatically on Node 22. SMTP port 465 is treated as secure TLS; port 587 uses the common STARTTLS-compatible configuration.

9. Improvements delivered over a basic/manual exception-management approach

- From fragmented follow-up to a single queue: exception data, ownership, aging and financial exposure are made visible in one workspace.
- From subjective prioritisation to explainable risk: the weighted score makes prioritisation repeatable and auditable.
- From delayed issue discovery to SLA-aware management: aging, deadline and breach status are calculated automatically.
- From isolated fixes to controlled resolution: validation, resolution notes, comments, timeline and audit entries preserve the operational record.
- From static status reporting to live decision support: dashboard and analytical views update from the same API used by the queue.
- From manual report assembly to self-service distribution: filtered PDF export and email report options are included in the workflow.
- From empty demo metrics to meaningful demo evidence: resolved seed records now power Average Resolution and Resolution Cycle Time.

- From a backend that exited on local start to a usable local service: the direct-run listener exposes the API on port 4004; the serverless export remains suitable for Netlify.
- From opaque simulated integrations to a safer control point: high-value SAP release requests require an explicit confirmation step.

10. Complete demonstration script

10.1 Start the application

Backend: Open a terminal in backend and run npm start. Confirm the message: Backend listening at http://localhost:4004.

Frontend: Open a second terminal in frontend and run npm run dev. Open the Vite URL displayed in the terminal.

Connection: The frontend development API base is http://localhost:4004/api. The Dashboard badge should change from Connecting to Live Data.

10.2 Suggested live demo flow

Step	Demonstrate	Key message to explain
1	Open Executive Overview.	Treasury managers immediately see count, exposure, SLA adherence, average resolution and risk/aging context.
2	Review recent high-priority exceptions.	Priority is calculated from risk, not manually sorted opinion.
3	Navigate to Exception Queue and filter by Critical/High or a document type.	Analysts narrow a cross-process portfolio to the work that matters now.
4	Open Assess on an exception.	Show score, rule breakdown, SLA deadline, AI explanation and recommended action.
5	Resolve an open exception with meaningful notes.	The system timestamps completion, captures the explanation and updates operational history.
6	Refresh Dashboard and Analytics.	Explain that newly resolved work affects count, average-resolution and cycle-time measures for the running session.
7	Use Export PDF after applying a filter.	The report is generated server-side from the same filtered data—no manual spreadsheet compilation.
8	Use Email Report or the detail email action.	Without SMTP it opens an Ethereal test preview; with SMTP configuration it delivers to a real inbox.
9	Open Financial Analytics.	Use category, partner, plant, recurrence, first-pass yield and revenue-at-risk insights to discuss root causes.
10	Optionally trigger SAP Release for a selected item.	Call out the high-value confirmation gate and that this environment simulates—not executes—a SAP transaction.

Demo talking points

- Start with business value: fewer missed exceptions, faster ownership and a clear view of cash/exposure risk.
- Then show operational control: validation, risk scoring, escalation, resolution notes and auditability.
- Close with management insight: KPIs, process deviations, export/email distribution and the SAP-ready integration boundary.

11. Security, quality and deployment no

Area	Current implementation	Production recommendation

Area	Current implementation	Production recommendation
Transport/API	Helmet, CORS, JSON size limit and rate limiting are enabled.	Restrict ALLOWED_ORIGIN, terminate TLS, add authentication/authorisation and central secrets…
Audit	Workflow actions append audit entries and timeline records in memory.	Persist immutable audit data; add retention, search and privileged review controls.
Data	Seeded JSON is copied into process memory at startup.	Use CAP persistence/HANA or an approved database; migrate seed/import flows to managed tables.
Email	Ethereal test mode or environment-provided SMTP.	Use organisation-approved SMTP/API provider, authenticated sender domain and delivery monitoring.
SAP	Mocked release call with a confirmation gate.	Use secure SAP destinations, OAuth/service credentials, real API contracts, retries and reconciliation.
Testing	32 unit checks pass for validation, risk and AI-explainer logic.	Add API integration, UI, security, performance and end-to-end deployment tests.

12. Known demo constraints

- Data is intentionally non-persistent. Restarting the backend reloads the eight seeded records and removes changes made during the session.
- The email feature is fully testable with Ethereal, but real external email requires SMTP credentials supplied by the organisation.
- The SAP function is a controlled simulation. It proves the workflow contract and payload shape but is not a live SAP posting/release.
- The CDS model is richer than the in-memory demo runtime; the next major step is wiring the API to CAP persistence and target SAP services.
- Potential extensions include SSO/role controls, attachment storage, asynchronous notifications, scheduled SLA escalation, Excel import UI, data retention and enterprise observability.

13. Verification record

The following implementation checks were completed during
32 of 32 tests; the frontend Vite production build completed s
application/pdf; and the email endpoint successfully generate
configured to listen on port 4004 when started directly, while
opening a second listener.

This report is generated from the implementation in this repo
stakeholder review and a clear handover of the current soluti